



Subtopic: Processor Design, Instruction Set Design, and Addressing

1. Processor Design

Definition

Processor design is the process of creating the **CPU's internal architecture**, deciding how it will execute instructions, handle data, and communicate with memory and I/O devices.

Key Aspects of Processor Design

1. Datapath Design:

- Determines how data flows inside the CPU.
- Includes ALU, registers, buses, and interconnections.

2. Control Unit Design:

- Directs execution of instructions by generating control signals.
- Two approaches:
 - *Hardwired Control*: Fixed logic circuits, fast but inflexible.
 - *Microprogrammed Control*: Uses microinstructions stored in control memory, slower but more flexible.

3. Pipeline Design:

- Breaks instruction execution into stages (fetch, decode, execute, memory access, write-back).
- Increases instruction throughput.

4. Parallelism:



- *Superscalar processors*: Execute multiple instructions per cycle.
- *Multicore processors*: Multiple CPUs integrated on one chip.

5. Performance Considerations:

- *Clock speed (GHz)*: Determines instruction execution rate.
 - *CPI (Cycles Per Instruction)*: Lower CPI means faster execution.
 - *Power efficiency*: Balancing performance with energy use.
-

2. Instruction Set Design

Definition

The **Instruction Set Architecture (ISA)** defines the set of machine-level instructions that the processor can execute. It acts as an **interface between hardware and software**.

Elements of Instruction Set Design

1. Instruction Types:

- *Data Processing*: Arithmetic and logic operations (ADD, SUB, AND).
- *Data Movement*: Load/Store instructions (MOV, LOAD, STORE).
- *Control Flow*: Branch, jump, procedure calls.
- *I/O Instructions*: Input/output handling.

2. Instruction Format:

- Specifies the layout of bits in an instruction.
- Common fields:



- *Opcode*: Operation to perform.
- *Operand(s)*: Data or address on which operation acts.
- *Mode bits*: Define addressing method.

3. Instruction Set Types:

- **RISC (Reduced Instruction Set Computer):**
 - Simple, uniform instructions.
 - Single-cycle execution.
 - Example: ARM, MIPS.
- **CISC (Complex Instruction Set Computer):**
 - Rich instruction set with complex operations.
 - Multiple cycles per instruction.
 - Example: Intel x86.

4. Design Considerations:

- *Simplicity*: Easier compiler design and faster execution.
- *Orthogonality*: Instructions should work uniformly with different data types and addressing modes.
- *Efficiency*: Minimize instruction length and execution time.

3. Addressing

Definition



Addressing refers to the **method of specifying the location of data (operands)** that an instruction will use during execution.

Types of Addressing Modes

1. Immediate Addressing:

- Operand is part of the instruction itself.
- Example: `ADD R1, #5` (Add 5 to register R1).
- Fastest but limited range.

2. Direct Addressing:

- Instruction specifies the **memory address** of the operand.
- Example: `LOAD R1, 1000` (Load data from memory address 1000).

3. Indirect Addressing:

- Instruction specifies a memory location that contains the address of the operand.
- Example: `LOAD R1, (1000)` (Fetch address stored at 1000, then load data from that address).

4. Register Addressing:

- Operand is stored in a register.
- Example: `ADD R1, R2` (Add contents of R2 to R1).

5. Register Indirect Addressing:

- Register contains the address of the operand.
- Example: `LOAD R1, (R2)` (Use R2 as a pointer to operand).



6. Indexed Addressing:

- Effective address = Base address + Index.
- Useful for accessing arrays.
- Example: `LOAD R1, 1000(R2)` (Load data from address = 1000 + contents of R2).

7. Relative Addressing:

- Effective address = Program Counter (PC) + offset.
- Used in branch instructions.
- Example: `JUMP +20` (Jump 20 bytes ahead of current PC).

✓ Summary

- **Processor Design** focuses on creating the CPU's datapath, control unit, and pipelines to achieve high performance.
- **Instruction Set Design (ISA)** defines the instructions that processors execute, balancing simplicity, efficiency, and compatibility (RISC vs CISC).
- **Addressing** determines how operands are located in memory or registers, with multiple addressing modes providing flexibility for different applications.